

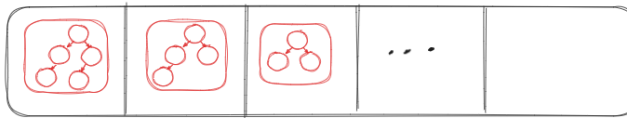
PROGRAMACION III

Lic. Aruquipa Marcelo

BOSQUES Múltiples Árboles

A la gestión de múltiples árboles en: vectores, pilas, colas, listas, ..., árboles

a) conjunto de árboles
ABN & ABB



b) conjunto de raíces
ABN & ABB

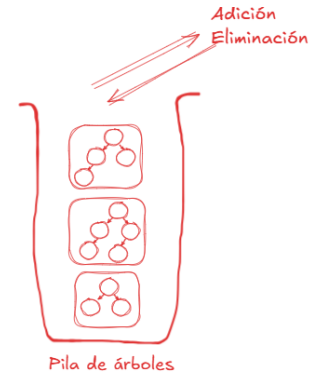
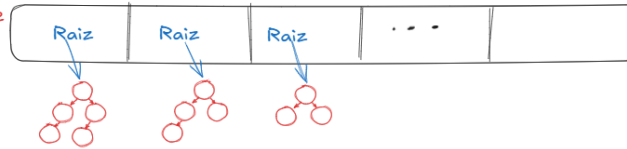


Diagrama de clases a)

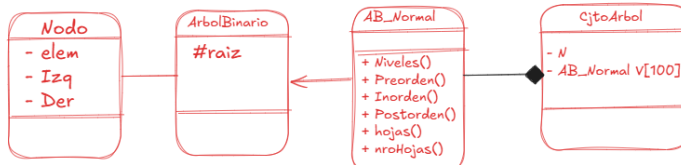
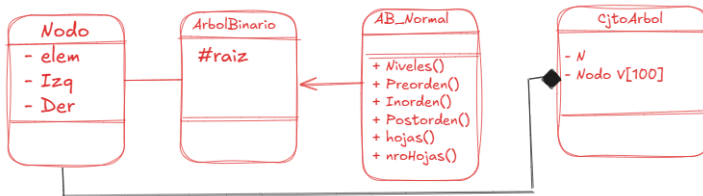


Diagrama de clases b)



Bosque

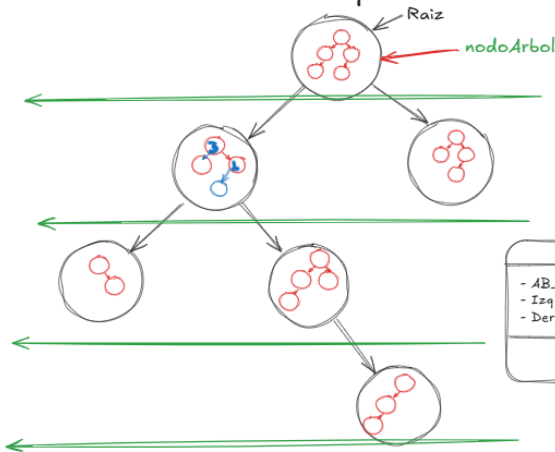
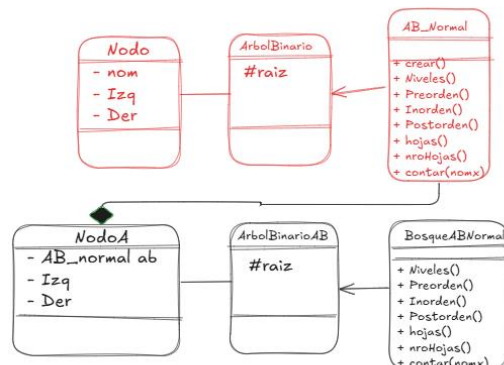
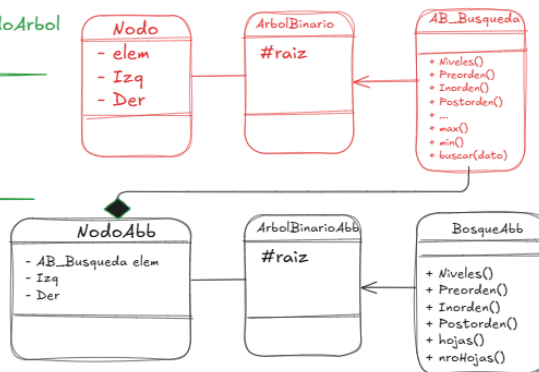


Diagrama de clases



Implementación

PROGRAMACION III

Lic. Aruquipa Marcelo

```
class BosqueABNormal : ArbolBinarioAB
{
    ....
    crear(NodoA r){
        if (r != null)
        {
            r.getAb().setRaiz(new Nodo);
            r.getAb().crear(r.getAb().getRaiz());
            print("tendra Izq ? s/n");
            read(res);
            if(res = "s"){
                nuevo <- new NodoA();
                r.setIzq(nuevo);
                crear(r.getIzq());
            }
            print("tendra Der? s/n");
            read(res);
            if(res = "s"){
                nuevo <- new NodoA();
                r.setDer(nuevo);
                crear(r.getDer());
            }
        }
    }
    niveles(){
        nivel, des <- new ColaSimpleA();
        nivel.adi(raiz)
        while(!nivel.esVacia()){
            while(!nivel.esVacia()){
                NodoA x <- nivel.eli();
                x.getAb().niveles(); //se muestra el arbol por niveles
                if(x.getIzq() != null)
                    des.adi(x.getIzq())
                if(x.getDer() != null)
                    des.adi(x.getDer());
            }
            nivel.vaciar(des);
        }
    }
    int contar(String nomx){
        nivel, des <- new ColaSimpleA();
        int total <- 0;
        nivel.adi(raiz)
        while(!nivel.esVacia()){
            while(!nivel.esVacia()){
                NodoA x <- nivel.eli();
                total <- total + x.getAb().contar(nomx);
                if(x.getIzq() != null)
                    des.adi(x.getIzq())
                if(x.getDer() != null)
                    des.adi(x.getDer());
            }
            nivel.vaciar(des);
        }
        return total;
    }
}
```

//suponemos implementado: recorridos en amplitud (niveles)
//y en profundidad, hojas, nroHojas, etc.

} //FIN class BosqueABNormal

```
main(){
    Z <- new BosqueABNormal();
    r <- new NodoA();
    Z.setRaiz(r);
    Z.crear(Z.getRaiz())
    Z.niveles();
    Z.preOrden();
    //Ejercicio. Dado un bosque de nombres de personas determinar el nro de veces
    //que aparece el nombre x
    print(Z.contar("rodrigo"));
}
```

```
class AB_Normal : ArbolBinario
{
    ....
    niveles(){
        nivel, des <- new ColaSimple();
        nivel.adi(raiz)
        while(!nivel.esVacia()){
            while(!nivel.esVacia()){
                Nodo x <- nivel.eli();
                print(x.getNom());
                //x.getPersona().mostrar();
                if(x.getIzq() != null)
                    des.adi(x.getIzq())
                if(x.getDer() != null)
                    des.adi(x.getDer());
            }
            nivel.vaciar(des);
        }
    }
    int contar(String nomx){
        nivel, des <- new ColaSimpleA();
        int cnt <- 0;
        nivel.adi(raiz)
        while(!nivel.esVacia()){
            while(!nivel.esVacia()){
                Nodo x <- nivel.eli();
                if(x.getNom() = nomx)
                    cnt++;
                if(x.getIzq() != null)
                    des.adi(x.getIzq())
                if(x.getDer() != null)
                    des.adi(x.getDer());
            }
            nivel.vaciar(des);
        }
        return cnt;
    }
}
```